

Министерство науки и высшего образования Российской Федерации
Московский физико-технический институт (государственный университет)

Физтех-школа прикладной математики и информатики
Кафедра дискретной математики

Выпускная квалификационная работа бакалавра

Решёточная и постквантовая криптография

Автор:

Студент Б05-228 группы
Савватеев Михаил Алексеевич

Научный руководитель:

Доктор физико-математических наук
Фролов Алексей Андреевич



Москва 2026

Аннотация

Решёточная и постквантовая криптография

Савватеев Михаил Алексеевич

Краткое описание задачи и основных результатов, мотивирующее
прочитать весь текст.

Abstract

Research and development of post-quantum cryptography methods

Содержание

1	Введение	4
2	Обозначения	6
3	Постановка задачи	8
4	Обзор существующих решений	10
5	Исследование и построение решения задачи	11
5.1	Свойства маппинга	11
5.2	Вычисление DFR	11
5.3	Границы на скорость кода	12
6	Описание практической части	14
6.1	Вычисление шума	14
6.2	Вероятности ошибок	15
6.3	Подсчёт t и R	16
7	Численные результаты	17
8	Заключение	20
	Приложение	23

1 Введение

Большая часть современных криптографических протоколов базируются на сложности задач факторизации натурального числа и дискретного логарифмирования элемента в заданной абелевой группе.

Таковыми являются, например, широко известная криптосистема RSA (*Rivest–Shamir–Adleman*), а также современные российские ГОСТы (*государственные стандарты*) для цифровой подписи документов и получения двумя сторонами общего секретного ключа.

При этом в 1994 - 1997 годах Шор разработал алгоритм на квантовом компьютере, который решает обе эти задачи в $\mathbb{N}$ за полиномиальное от длины числа время [2]. В дальнейшем алгоритм улучшался и модифицировался, в том числе, в 2003 году была опубликована и подробно описана его версия на эллиптических кривых [3].

Сейчас многими компаниями и государствами ведутся исследования по построению мощных стабильных квантовых компьютеров. В случае успеха оных все вышеупомянутые протоколы окажутся уязвимыми.

В связи с этим, начиная с 2016 - 2017 годов NIST (*National Institute of Standards and Technology в США*) проводит конкурс по отбору лучших протоколов постквантовой (*устойчивой к атакам в том числе на квантовом компьютере*) криптографии.

Немалая доля из них основана на решёточной криптографии вообще и **LWE**-(*Learning With Errors*)-problem в частности [4]. Одним из главных преимуществ этих криптосистем является наличие **ACH** (*Average-Case*), а не **WCH** (*Worst-Case*) сложности (*Hardness*).

Неформально говоря, это означает, что трудными для взлома являются случайные параметры протокола, а не худшие из всех возможных. В реальных криптосистемах данное свойство очень полезно, поскольку генерация рандомных бит при шифровании необходима.

К сожалению, протоколы, основанные **LWE**-problem, хотя являются простыми и понятными, требуют довольно много вычислений. Поэтому с

целью оптимизации в статье [6] вводится **RLWE**-(*Ring LWE*)-problem. Это модификация **LWE** в многочленах над $\mathbb{Z}_q$ с ускоренными вычислениями за счёт **NTT** (*Number Theoretic Transform*).

Более подробно все эти и некоторые дальнейшие концепции описаны в вышедшем в 2022 году обзоре [1]. В моей же работе я буду использовать и улучшать результаты следующей статьи [10].

В ней процесс **RLWE**-шифрования рассматривается, как передача данных по зашумлённому **RLWE**-каналу. Исследуются свойства этого канала, описывается, как увеличить скорость передачи и при этом снизить **DFR** (*Decryption Failure Probability/Rate*) при расшифровке сообщения.

В частности, для уменьшения **DFR** при фиксированном **CFR** (*Coefficient Failure Rate*) используются **ECC** (*Error Correcting Codes*). Эта конструкция позволяет исправлять вплоть до t покоординатных ошибок при передаче вектора-сообщения фиксированной длины n (здесь n, t — некоторые выбираемые исследователем параметры).

При этом измерять кол-во произошедших ошибок можно по-разному. В моей работе я пробую разные способы (метрика Хэмминга, метрика Ли) и вычисляю, как изменяется от этого скорость **RLWE**-канала.

Как и в статье [10], я варьирую размер алфавита Q , которому принадлежат символы сообщения, и наблюдаю, как изменяются от этого оптимальные параметры канала.

Для этого я выбираю несколько конкретных криптосистем (**LAC 256**, **NewHope 1024**), целевое **DFR** и исследую их. В дальнейшем реализованный код и применённые методы можно обобщить также на другие криптосистемы с иными размерами алфавита и метриками.

2 Обозначения

Под $\mathbb{Z}_m$ будем подразумевать кольцо $\mathbb{Z}/m\mathbb{Z}$ с обычными представителями $\{0, \dots, m-1\}$. Ими же зададим стандартное вложение $\mathbb{Z}_m \hookrightarrow \mathbb{Z}$, сравнение из которого будем использовать, говоря $x < y$ для $x, y \in \mathbb{Z}_m$.

Пусть $n \in \mathbb{N}$. Обозначим за $\mathcal{R}^n$ кольцо многочленов $\mathbb{Z}[x]/(x^n + 1)$. Вспомним, что, если n — степень 2, оно является полем, ведь $x^n + 1$ будет многочленом деления круга $\implies$ неприводимым над $\mathbb{Z}$. Если дополнительно число q — простое, примем за $\mathcal{R}_q^n$ кольцо $\mathbb{Z}_q[x]/(x^n + 1)$.

Элементом $\mathcal{R}_q^n$ естественным образом сопоставляются многочлены с n коэффициентами из $\mathbb{Z}_q$ степени $n-1$. Будем писать их жирным шрифтом $\mathbf{a}$. При этом их i -ый коэффициент будем обозначать a_i . Если все коэффициенты независимо генерируются из распределения P или равномерно из мн-ва S , будем обозначать это $\mathbf{a} \leftarrow P$ или $\mathbf{a} \leftarrow S$ соответственно.

Сумма и произведение естественным образом наследуются в $\mathcal{R}_q^n$ из $\mathbb{Z}_q[x]$, причём из-за выполнения в нём $x^n = -1$ получаем равенство:

$$(\mathbf{ab})_k = \sum_{i=0}^k a_i b_{k-i} - \sum_{i=k+1}^{n-1} a_i b_{n+k-i}$$

Для распределения P будем обозначать вероятность исхода x , как $P[x]$. Если при этом P принимает значения в $\{0, \dots, d-1\}$, ему естественным образом сопоставляется многочлен:

$$\hat{P}(z) = \sum_{i=0}^{d-1} P[i] z^i$$

Для события A примем за $\Pr(A)$ его вероятность, а за $I\{A\}$ — индикатор его выполнения.

Станем обозначать биномиальное распределение с n попытками и вероятностью успеха p , как $\mathcal{B}(n, p)$, оно принимает значения $0 \leq x \leq n$. Если $\mu, \nu \sim \mathcal{B}(k, \frac{1}{2})$ — независимые случайные величины, распределение разности $\mu - \nu$ обозначим за χ_k (*centered binomial distribution*), соответственно, его значения будут $-k \leq x \leq k$.

Свёртку в $\mathbb{Z}_q$ распределений P и R будем писать, как $P * R$. Свёртку же распределения P с собой $n - 1$ раз изобразим $P^{\otimes n}$.

Под алфавитом размера Q будем подразумевать $\Sigma_Q = \{0, \dots, Q - 1\}$. Введём на Σ_Q норму Хэмминга $|s|_H$ и норму Ли $|s|_L$, как:

$$|s|_H = I\{s \neq 0\}; \quad |s|_L = \min(s, Q - s)$$

После этого естественным образом определим норму на Σ_Q^n :

$$\|m\| = \sum_{i=0}^{n-1} |m_i|$$

3 Постановка задачи

Пусть $2 \leq Q \leq q$. Определим отображение $\text{Map} : \Sigma_Q \rightarrow \mathbb{Z}_q$, как:

$$\text{Map}(s) = \left\lfloor \frac{sq}{Q} \right\rfloor$$

В обратную сторону, положим $\text{Demap} : \mathbb{Z}_q \rightarrow \Sigma_Q$, как $\text{Demap}(x)$ — прообраз ближайшего к x по метрике Ли в $\mathbb{Z}_q$ остатка из $\text{Map}(\Sigma_Q)$. Если таких несколько, выберем минимальный из них.

После этого сообщение $m \in \Sigma_Q^n$ станем кодировать и декодировать посимвольно $\text{Map} : \Sigma_Q^n \rightarrow \mathcal{R}_q^n$ и $\text{Demap} : \mathcal{R}_q^n \rightarrow \Sigma_Q^n$.

Теперь можно описать протокол для передачи сообщений:

Алгоритм 1: Генерация ключей

Вход: $q, k \in \mathbb{N}$

$$\mathbf{a} \leftarrow \mathbb{Z}_q$$

$$\mathbf{s}, \mathbf{e} \leftarrow \chi_k$$

$$\mathbf{b} = \mathbf{a}\mathbf{s} + \mathbf{e}$$

Выход: $pk = (\mathbf{b}, \mathbf{a}), sk = \mathbf{s}$

Алгоритм 2: Шифрование

Вход: $m \in \Sigma_Q^n, pk$

$$\mathbf{s}', \mathbf{e}', \mathbf{e}'' \leftarrow \chi_k$$

$$\mathbf{u} = \mathbf{a}\mathbf{s}' + \mathbf{e}'$$

$$\mathbf{v} = \mathbf{b}\mathbf{s}' + \mathbf{e}'' + \text{Map}(m)$$

Выход: $c = (\mathbf{u}, \mathbf{v})$

Алгоритм 3: Разшифровывание

Вход: c, sk

$$\tilde{m} = \text{Demap}(\mathbf{v} - \mathbf{u}\mathbf{s})$$

Выход: $\tilde{m}$

При этом шумом $\mathbf{z}$ назовём $\mathbf{v} - \mathbf{u}\mathbf{s} - \text{Map}(m)$. Имеем:

$$\mathbf{z} = (\mathbf{b}\mathbf{s}' + \mathbf{e}'') - (\mathbf{a}\mathbf{s}' + \mathbf{e}')\mathbf{s} = \mathbf{e}\mathbf{s}' - \mathbf{e}'\mathbf{s} + \mathbf{e}''$$

Заметим, что событие $\tilde{m} \neq m \iff \text{Демар}(\mathbf{z}) \neq 0^n$. Таким образом, можно рассматривать процесс зашифровывания, как **RLWE**-канал, который при передаче по нему сообщения $\text{Мар}(m)$ из $\mathcal{R}_q^n$ добавляет к нему шум $\mathbf{z}$ также из $\mathcal{R}_q^n$.

За сложность взлома протоколов из данного семейства отвечают некоторые хорошие свойства $\mathcal{R}_q^n$, изученные

4 Обзор существующих решений

Есть несколько широко известных WCH проблем на решётках: SVP (*Shortest Vector Problem*), GapSVP, SIVP (*Shortest independent vectors problem*) и т.п. В 1996 году Ажтай свёл часть из них к ACH проблеме SIS (*Short Integer Solution*) [5]. Позже Регев проделал аналогичную вещь с LWE.

5 Исследование и построение решения задачи

5.1 Свойства маппинга

Пусть $x \in \mathbb{N}$. Представим xq в виде $Qm + r$ для $0 \leq r < Q$.

$$\left\lfloor \frac{(x+1)q}{Q} \right\rfloor = \left\lfloor \frac{Qm + r + q}{Q} \right\rfloor = m + \left\lfloor \frac{q+r}{Q} \right\rfloor$$

Получаем, что длины $\text{Map}(x+1) - \text{Map}(x)$ отличаются на ≤ 1 , так как

$$\left\lfloor \frac{xq}{Q} \right\rfloor = m; \quad \left\lfloor \frac{q}{Q} \right\rfloor \leq \left\lfloor \frac{q+r}{Q} \right\rfloor \leq \left\lceil \frac{q}{Q} \right\rceil$$

При этом Декар (с точностью до округлений) отправляет в x остатки со 2-ой половины предыдущего $[\text{Map}(x-1), \text{Map}(x)]$ и 1-ой половины следующего $[\text{Map}(x), \text{Map}(x+1)]$ за ним отрезков.

Из доказанного выше свойства про их длины, если зафиксировать символ s , в него точно декодируются все x вида:

$$|x - \text{Map}(s)| < \frac{1}{2} \cdot \left\lfloor \frac{q}{Q} \right\rfloor \iff |x - \text{Map}(s)| < \left\lfloor \frac{q}{2Q} \right\rfloor$$

5.2 Вычисление DFR

Пусть $\mu, \nu \sim \chi_k$ — независимые случайные величины в $\mathbb{Z}_q$. Обозначим распределение их произведения $\mu \cdot \nu$ за ξ_k . Положим $\psi = \xi_k^{\otimes n} * \xi_k^{\otimes n} * \chi_k$.

В статье [10] доказывалось, что это распределение коэффициентов $\mathbf{z}$. Таким образом, полагая их независимыми, получаем $\mathbf{z} \leftarrow \psi$, и

$$b = \left\lfloor \frac{q}{2Q} \right\rfloor; \quad \text{CFR} \leq 1 - \sum_{i=1-b}^{b-1} \psi[i]$$

Математически предположение это неверно, однако широко распространено в силу своей удобства и хорошо себя зарекомендовало на практике.

В итоге, по формулам бинома Ньютона в метрике Хэмминга:

$$\text{DFR} = \sum_{i=t+1}^n C_n^i \cdot \text{CFR}^i \cdot (1 - \text{CFR})^{n-i}$$

Теперь проанализируем, что происходит в метрике Ли. Мы умеем восстанавливать $\tilde{m}$ с $\|\tilde{m} - m\|_L \leq t$. Назовём $x_i = |(\tilde{m} - m)_i|_L$.

Обозначим кроме того $ch : \mathbb{Z}_Q \rightarrow \mathbb{Z}$ функцию вида:

$$ch(x) = \max_{s \in \Sigma_Q} \{ |\text{Demap}(\text{Map}(s) + x) - s|_L \}$$

Нам подходят вектора $\tilde{m} - m = (x_1, \dots, x_n)$ с $x_1 + \dots + x_n \leq t$. По ψ можно вычислить $\Pr(x_i = k)$ для $0 \leq k < Q$. Более конкретно, пусть

$$\sigma[m] = \sum_{ch(x)=m} \psi[x]$$

После этого можно вычислить DFR через производящие функции:

$$P(z) = \sum_{m=0}^{Q-1} \sigma[m] z^m; \quad \text{DFR} = \sum_{i=t+1}^{(Q-1)n} P^n[i]$$

Заметим, что то же самое работает и для метрики Хэмминга, если взять её $P(z) = (1 - \text{CFR}) + \text{CFR} \cdot z$.

5.3 Границы на скорость кода

Умея считать оценку на DFR при данном t , можно подобрать необходимое кол-во исправлений, чтобы вероятность ошибки была меньше порогового значения.

Примем $V(r)$ — объём шара радиуса r . В Q -ичном алфавите по метрике Хэмминга есть стандартная формула:

$$V_H(r) = \sum_{i=0}^r C_n^i (Q-1)^i;$$

В метрике Ли же всё немного хитрее. Для $0 \leq i < Q$ положим:

$$c_i = |\{s \in \Sigma_Q : |s|_L = i\}| = 2 - I\{i = 0 \vee 2i = Q\}$$

После этого снова воспользуемся производящими функциями:

$$P(z) = \sum_{m=0}^{Q-1} c_m z^m; \quad V_L(r) = \sum_{i=0}^r P^n[i]$$

Опять же, для метрики Хэмминга формула выше получается, если взять $P(z) = 1 + (Q-1)z$.

После этого по кодовому расстоянию $d = 2t + 1$ и размерности кода n можно посчитать границы Хэмминга [7] и Варшамова-Гилберта [8], [9] на его размер M :

$$M_H \leq \left\lfloor \frac{Q^n}{V(t)} \right\rfloor; \quad M_{GV} \geq \left\lceil \frac{Q^n}{V(d-1)} \right\rceil$$

Наконец, можно вычислить границы на скорость оптимального кода:

$$R = \frac{\log(M)}{n}$$

6 Описание практической части

Для написания кода использовался язык **Python** и опирающийся на его синтаксис **SageMath**. В качестве криптографических протоколов, основываясь на статье [10], были выбраны **LAC** и **NewHope**.

Зафиксированы следующие наборы параметров:

Протокол	n	q	k
LAC	1024	251	1
NewHope	1024	12289	8

6.1 Вычисление шума

Вероятности исходов $\mathcal{B}(k, \frac{1}{2})$ посчитаны по формулам через модуль `scipy.stats`. После этого распределение χ_k вычислено за время $O(k^2)$ по определению: перебором возможных значений (μ, ν) , вычислением значения $\mu - \nu$ и прибавлением к нему вероятности текущего исхода.

В распределении $\mathcal{B}(k, p)$ присутствуют значения $0 \leq x \leq k$, отсюда в χ_k они будут $-k \leq x \leq k$ — всего $O(k)$ штук. Таким образом, ξ_k также вычисляется по определению за $O(k^2)$ тем же способом.

После этого нам нужно свернуть ξ_k само с собой $2n$ раз, а после этого ещё 1 раз с χ_k для получения распределение шума ψ . Все операции происходят в $\mathbb{Z}_q$, поэтому на каждом шаге распределения принимают не больше q разных значений $0 \leq x < q$.

Сворачивать с собой ξ_k будем, используя алгоритм бинарного возведения в степень и тратя на это $O(\log(n))$ шагов. Во время каждой его итерации, а также в конце при свёртке с χ_k нам нужно находить свёртку распределений со значениями из $\{0, \dots, q-1\}$.

Обозначая их P и Q , значения $P * Q$ ищутся в $\mathbb{Z}_q$ по формуле:

$$(P * Q)[y] = \sum_{x=0}^y P[x]Q[y-x] + \sum_{x=y+1}^{q-1} P[x]Q[q+y-x]$$

Если взять проассоциированные с распределениями многочлены, свёрт-

ка выражается через коэффициенты их произведения:

$$(P * Q)[y] = (\hat{P}\hat{Q})[y] + (\hat{P}\hat{Q})[q + y]$$

Но произведение многочленов длины q можно с помощью **НТТ** искать за время $O(q \log(q))$. В моём коде для этих целей я использую функцию `fftconvolve` из модуля `scipy.signal`.

В итоге, распределение ψ может быть посчитано с использованием $O(q \log(q) \log(n) + k^2)$ времени и $O(q \log(n))$ памяти, чего вполне хватает для большинства используемых сейчас криптосистем.

6.2 Вероятности ошибок

Дальше, для быстрого вычисления Демар предподсчитаем его значения. Будем перебирать символ $x \in \mathbb{Z}_q$ и смотреть, к образу какого из $s \in \Sigma_Q$ он ближе — асимптотика $O(qQ)$.

Теперь Мар и Демар считаются за $O(1)$, и можно по определению вычислить значения ch — также за $O(qQ)$.

Зная ψ , по ним за $O(q)$ восстанавливается распределение σ . Наконец, для оценки **DFR** у нас снова возникает задача посчитать P^n , но уже в $\mathbb{R}[x]$.

Посмотрим на устройство алгоритма бинарного возведения в степень. Так как $\deg(P) = Q - 1$ и, следовательно, $\deg(P^m) \leq (Q - 1)m$, можно дополнять P^m нулями до длины Qm .

В процессе вычисления мы ≤ 2 раза перемножим многочлены длины $\frac{1}{2}Qn$, ≤ 2 раза — длины $\frac{1}{4}Qn$ и т.д. С **НТТ** суммарно получается:

$$Time \leq \sum_{i=1}^B \frac{1}{2^i} Qn \cdot \log \left(\frac{1}{2^i} Qn \right)$$

Огрубляя и оценивая сумму геометрической прогрессии, выводим:

$$Time \leq Qn \log(Qn)$$

Имеем асимптотику $O(Q(n \log(Qn) + q))$ времени и $O(q + Qn)$ памяти.

6.3 Подсчёт t и R

Значение DFR при фиксированном n убывает при возрастании t . Отсюда первый момент выполнения $\text{DFR} < 10^{-6}$ можно искать с помощью алгоритма бинарного поиска.

Вычисление DFR происходит суммированием за $O(Qn)$, значение t ограничено тем же числом — асимптотика $O(Qn \log(Qn))$.

Дальше по данным t и n нужно вычислить M . В случае Хэмминга я применяю для этого методы `gilbert_lower_bound` и `hamming_upper_bound` из библиотеки `sage.codes.bounds`.

В метрике Ли я явно строю P , как элемент `PolynomialRing(ZZ)` из `SageMath`, и после встроенным методом считаю P^n , откуда узнаю объёмы шаров и границы на M соответственно.

Данный шаг снова работает за асимптотику $O(Qn \log(Qn))$. Наконец, чтобы узнать R , я рассматриваю M , как элемент `RealField(100)` из `SageMath`, и встроенной функцией нахожу его логарифм.

7 Численные результаты

Вот результаты для **LAC** и **NewHope**. Параметры $[n, q, k]$ были взяты $[1024, 251, 1]$ и $[1024, 12289, 8]$ соответственно. Целевой DFR положим 10^{-6} .

Для **LAC** имеет смысл рассматривать размер алфавита Q от 2 до 6 символов включительно. Посчитав распределение в $\mathbb{Z}_q$ шума на компьютере, бинпоиском подбираем необходимое кол-во исправляемых ошибок.

Q	t	err
2	21	0.0058
3	109	0.0668
4	235	0.1709
5	345	0.2695
6	448	0.3651

Hamming

Q	t	err1	err2
2	21	0.0058	0
3	109	0.0668	0
4	235	0.1709	3.64e-05
5	346	0.2686	0.0009
6	456	0.3594	0.0058

Li

Последний столбец отвечает за вероятность ошибки в каждом символе (если рассматривать их независимо). В метрике Ли это вероятности отличия от исходного символа на расстояние 1 и 2 соответственно.

Для **NewHope** делаем то же самое, только рассматривая другие Q .

Q	t	err
2	0	≈ 0
3	0	≈ 0
4	0	≈ 0
5	0	1.52e-11
6	1	1.71e-08
7	1	1.32e-06
8	3	2.28e-05
9	5	1.66e-04

Hamming, Li

Для маленьких размеров алфавита вероятность двойной ошибки пренебрежимо мала, так что результаты одинаковы.

Q	t	err
10	7	0.0007
12	18	0.0047
14	38	0.0155
16	65	0.0339
18	100	0.0596
20	138	0.0899
23	199	0.1402
26	259	0.1923
30	333	0.2585
34	398	0.3186
38	455	0.3722

Hamming

Q	t	err1	err2
10	7	0.0007	≈ 0
12	18	0.0047	≈ 0
14	38	0.0155	≈ 0
16	65	0.0339	2.32e-10
18	100	0.0596	1.74e-08
20	138	0.0899	3.87e-07
23	199	0.1402	1.00e-05
26	259	0.1922	0.0001
30	334	0.2578	0.0007
34	402	0.3158	0.0028
38	466	0.3647	0.0075

Li

Тем не менее, при больших значениях Q разница проявляется. Далее по известным n и d можно посчитать границы на скорость R .

Q	R_{GV}	R_H
2	0.7569	0.8591
3	0.6298	0.9939
4	0.2821	0.8639
5	0.0674	0.7312
6	0.0010	0.5854

LAC, Hamming

Q	R_{GV}	R_H
2	0.7569	0.8591
3	0.6298	0.9939
4	0.4507	0.9767
5	0.3495	0.9804
6	0.3072	0.9879

LAC, Li

Видно, что при маленьких Q и t разница между метриками Хэмминга и Ли небольшая или вообще отсутствует, при увеличении же этих параметров Хэмминг начинает резко убывать, а Ли почти не уменьшается (за счёт гораздо меньшего объёма шаров, возникающих вокруг кодовых слов).

Q	R_{GV}	R_H
2	1.0000	1.0000
3	1.5850	1.5850
4	2.0000	2.0000
5	2.3219	2.3219
6	2.5619	2.5729
7	2.7838	2.7951
8	2.9342	2.9650
9	3.0643	3.1132
10	3.1775	3.2439
12	3.2475	3.3999
14	3.1555	3.4450
16	2.9596	3.4151
18	2.6640	3.3137
20	2.3412	3.1837
23	1.8315	2.9513
26	1.3565	2.7150
30	0.8186	2.4222
34	0.4060	2.1680
38	0.1187	1.9473

NewHope, Hamming

Q	R_{GV}	R_H
2	1.0000	1.0000
3	1.5850	1.5850
4	2.0000	2.0000
5	2.3219	2.3219
6	2.5645	2.5742
7	2.7868	2.7966
8	2.9448	2.9703
9	3.0838	3.1230
10	3.2070	3.2587
12	3.3331	3.4429
14	3.3518	3.5442
16	3.3163	3.5966
18	3.2367	3.6080
20	3.1565	3.6074
23	3.0427	3.5936
26	2.9581	3.5841
30	2.8863	3.5841
34	2.8495	3.5972
38	2.8290	3.6139

NewHope, Li

Также интересно, что вероятность возникновения отличия на ≥ 2 в метрике Ли перестаёт быть пренебрежительно мала лишь к моменту, когда вероятность отличия на 1 становится огромна (порядка $\frac{1}{4}$ и выше).

Таким образом, пока кол-во исправляемых ошибок остаётся не слишком большим, можно считать их все переходами из символа в соседний.

8 Заключение

Здесь надо перечислить все результаты, полученные в ходе работы. Из текста должно быть понятно, в какой мере решена поставленная задача.

Список литературы

- [1] *Li, Yang*. A Tutorial Introduction to Lattice-based Cryptography and Homomorphic Encryption / Yang Li, Kee Siong Ng, Michael Purcell // *ArXiv*. — 2022. — Vol. abs/2208.08125.
- [2] *Shor, Peter W*. Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer / Peter W. Shor // *SIAM Journal on Computing*. — 1997. — Vol. 26, no. 5. — P. 1484–1509.
- [3] *Proos, John*. Shor’s discrete logarithm quantum algorithm for elliptic curves / John Proos, Christof Zalka // *Quantum Info. Comput.* — 2003. — Vol. 3, no. 4. — P. 317–344.
- [4] *Regev, Oded*. On lattices, learning with errors, random linear codes, and cryptography / Oded Regev // Proceedings of the Thirty-Seventh Annual ACM Symposium on Theory of Computing. — STOC ’05. — New York, NY, USA: Association for Computing Machinery, 2005. — P. 84–93.
- [5] *Ajtai, M*. Generating hard instances of lattice problems (extended abstract) / M. Ajtai // Proceedings of the Twenty-Eighth Annual ACM Symposium on Theory of Computing. — STOC ’96. — New York, NY, USA: Association for Computing Machinery, 1996. — P. 99–108.
- [6] *Lyubashevsky, Vadim*. On Ideal Lattices and Learning with Errors over Rings / Vadim Lyubashevsky, Chris Peikert, Oded Regev // *J. ACM*. — 2013. — Vol. 60, no. 6. — 35 pp.
- [7] *Hamming, R. W*. Error Detecting and Error Correcting Codes / R. W. Hamming // *Bell System Technical Journal*. — 1950. — Vol. 29, no. 2. — Pp. 147–160.
- [8] *Gilbert, E. N*. A comparison of signalling alphabets / E. N. Gilbert // *The Bell System Technical Journal*. — 1952. — Vol. 31, no. 3. — Pp. 504–522.

- [9] *Varshamov, R. R.* Estimate of the number of signals in error correcting codes / R. R. Varshamov // *Doklady Akad. Nauk, S.S.S.R.* — 1957. — Vol. 117. — Pp. 739–741.
- [10] *Maringer, Georg.* Higher Rates and Information-Theoretic Analysis for the RLWE Channel / Georg Maringer, Sven Puchinger, Antonia Wachter-Zeh // 2020 IEEE Information Theory Workshop (ITW). — 2021. — Pp. 1–5.
- [11] Coding with Cyclic PAM and Vector Quantization for the RLWE/MLWE Channel / Irina E. Bocharova, Henk D. L. Hollmann, Karan Khathuria et al. // 2022 IEEE International Symposium on Information Theory (ISIT). — 2022. — Pp. 666–671.

Приложение

Здесь необходимо написать приложение, которое вы должны придумать самостоятельно.